Reading Coverage Reports: The coverage_reports Demo

SE 413/513 Software Testing · Demo: demos/coverage_reports

Dr. Kamrul Hasan, Grand Valley State University

Learning objectives

1

Reproduce

Run the demo's suites under coverage.py and pytest-cov, and reproduce each report.

2

Read

Read every column of a report, including line ranges and arrows such as 16->18.

3

Close the gaps

Turn each gap into a behavior-level test, and explain why 100% is still not “fully tested.”

What is in the demo

LIVE DEMO 1
setup + look around

File	What it shows	Branch cov.
`checkout_app/checkout.py`	The code under test (don't reformat it)	n/a
`tests/test_checkout_v1.py`	Thin first suite: one happy path per function	66%
`tests/test_checkout_coupons_only.py`	Only two coupons; produces an arrow	41%
`tests/test_checkout_v2.py`	Every gap closed with a real assertion	100%
`tests/test_checkout_noassert.py`	Runs everything, checks nothing	100%
`coverage_settings.toml`	Saved coverage settings	n/a

Setup

```
cd demos/coverage_reports

python -m pip install \
  pytest pytest-cov
```

Run from this folder

Report paths and line numbers then match the slides exactly.

The code under test

checkout_app/checkout.py (blank lines hidden; numbers are real)

```
 1  """A small checkout module ..."""
 4  def subtotal(items):
 5      total = 0.0
 6      for price, qty in items:
 7          if qty < 0:
 8              raise ValueError("quantity ...")
 9          total += price * qty
10      return total
13  def apply_coupon(amount, code):
14      if code == "SAVE10":
15          amount = amount * 0.90
16      elif code == "FLAT5":
17          amount = max(amount - 5, 0)
18      return round(amount, 2)
21  def shipping_fee(amount, is_member):
22      if is_member or amount >= 50:
23          return 0.0
24      return 7.99
27  def format_receipt(amount):
28      return f"Total due: ${amount:.2f}"
```

Red: never run by v1. **Amber**: decision went only one way.

tests/test_checkout_v1.py

```
def test_subtotal_two_items():
    assert subtotal(
      [(10.0, 2), (5.0, 1)]) == 25.0

def test_save10_coupon():
    assert apply_coupon(
      100.0, "SAVE10") == 90.0

def test_member_ships_free():
    assert shipping_fee(
      20.0, True) == 0.0
```

All three tests pass. How much of checkout.py did they exercise?

Measuring statement coverage

LIVE DEMO 2
81%, then 74%

Without --source: test files counted

```
coverage run -m pytest tests/test_checkout_v1.py
coverage report
```

```
checkout_app/checkout.py      19    5   74%
tests/test_checkout_v1.py      7    0  100%
TOTAL                         26    5   81%
```

81%

inflated by the test file

74%

checkout.py, statements only

With --source and -m: application code only

```
coverage run --source=checkout_app \
  -m pytest tests/test_checkout_v1.py
coverage report -m
```

```
checkout.py   19   5   74%   8, 16-17, 24, 28
```

Missing	Behavior never checked
8	Negative quantity is rejected
16-17	The FLAT5 coupon
24	Non-members pay shipping
28	Receipt formatting

Try it: reproduce and translate

TRY IT 1
students · 5 min

1 Reproduce the 81% and 74% reports. Which file inflates the first total?

```
coverage erase
coverage run -m pytest tests/test_checkout_v1.py
coverage report
coverage run --source=checkout_app \
  -m pytest tests/test_checkout_v1.py
coverage report -m
```

2 For each Missing entry, write one sentence: what behavior did no test check?

```
Missing: 8, 16-17, 24, 28

8     -> ...
16-17 -> ...
24    -> ...
28    -> ...
```

Done early? Run coverage report --skip-covered. Which rows disappear, and when would that help?

Turning on branch coverage

LIVE DEMO 3
74% drops to 66%

```
coverage run --branch --source=checkout_app -m pytest tests/test_checkout_v1.py
coverage report -m
```

```
Name                        Stmts   Miss  Branch  BrPart  Cover   Missing
checkout_app/checkout.py       19      5      10       3    66%   8, 16-17, 24, 28
```

Column	Meaning	v1 value
`Branch`	Decision outcomes (two per decision)	10: lines 6, 7, 14, 16, 22
`BrPart`	Decisions that went only one way	3: lines 7, 14, 22
`Cover`	Statements + branch outcomes combined	66%

Same tests, lower number: the report now counts decision outcomes it ignored before. Line 16 is missing, not partial, because it never ran at all.

How 66% is computed

$$(14 + 5) \;/\; (19 + 10)$$

$= 19 / 29 \approx 65.5\% \rightarrow 66\%$

14 of 19 statements ran; 5 of 10 branch outcomes were taken.

Reading an arrow: 16->18

LIVE DEMO 4
the 16->18 arrow

test_checkout_coupons_only.py tests only the SAVE10 and FLAT5 coupons.

```
coverage run --branch --source=checkout_app -m pytest tests/test_checkout_coupons_only.py
coverage report -m
```

```
checkout_app/checkout.py     19     10      10       1    41%   5-10, 16->18, 22-24, 28
```

8	line 8 never ran
16-17	lines 16 through 17 never ran
16->18	16 and 18 both ran, but never one straight after the other

Every arrow is a question

What input makes line 16 go straight to line 18?

Neither coupon matches: an unrecognized code. That is the missing test.

Try it: from arrow to test

TRY IT 2
students · 6 min

1. Run the coupons-only suite. Check 41% with the formula (statements run + outcomes taken) / 29.

```
coverage erase
pytest tests/test_checkout_coupons_only.py \
  --cov=checkout_app --cov-branch \
  --cov-report=term-missing
```

2. Write tests/test_mine.py with one test that removes 16->18. Predict the new Cover first.

```
from checkout_app.checkout import apply_coupon

def test_unknown_code():
    assert apply_coupon(...) == ...
```

3. Run both files together and check your prediction. Then delete test_mine.py.

```
pytest tests/test_checkout_coupons_only.py \
  tests/test_mine.py --cov=checkout_app \
  --cov-branch --cov-report=term-missing
rm tests/test_mine.py
```

Same numbers with pytest-cov, plus HTML

LIVE DEMO 5
pytest-cov + HTML

```
pytest tests/test_checkout_v1.py --cov=checkout_app \
  --cov-branch --cov-report=term-missing
```

```
checkout.py   19   5   10   3   66%   8, 16-17, 24, 28
3 passed
```

```
pytest tests/test_checkout_v1.py --cov=checkout_app \
  --cov-branch --cov-fail-under=80
```

```
FAIL Required test coverage of 80% not reached.
  Total coverage: 65.52%
3 passed                                    exit 1
```

All tests passed, yet the command fails. That nonzero exit is what fails a CI build.

coverage html → htmlcov/index.html



Green bar

line ran

Red background

line never ran

Yellow background

partial branch; hover to see which jump was missed

Index page: statements 74%, branches 50%, total 66%.

Closing the gaps

TRY IT 3
students · 6 min

Gap in v1	Behavior never checked	New test in v2
`8`	Negative quantity is rejected	??
`16-17`	FLAT5 subtracts $5	??
`17 (max)`	FLAT5 never goes below 0	??
`16->18`	Unknown code changes nothing	??
`24`	Non-member small order pays	??
`28`	Receipt formatting	??

Closing the gaps

TRY IT 3
students · 6 min

Gap in v1	Behavior never checked	New test in v2
`8`	Negative quantity is rejected	`subtotal([(10.0, -1)]) raises ValueError`
`16-17`	FLAT5 subtracts $5	??
`17 (max)`	FLAT5 never goes below 0	??
`16->18`	Unknown code changes nothing	??
`24`	Non-member small order pays	??
`28`	Receipt formatting	??

Closing the gaps

LIVE DEMO 6
v2 suite: 100%

Gap in v1	Behavior never checked	New test in v2
`8`	Negative quantity is rejected	`subtotal([(10.0, -1)]) raises ValueError`
`16-17`	FLAT5 subtracts $5	`apply_coupon(100.0, "FLAT5") == 95.0`
`17 (max)`	FLAT5 never goes below 0	`apply_coupon(3.0, "FLAT5") == 0.0`
`16->18`	Unknown code changes nothing	`apply_coupon(100.0, "BOGUS") == 100.0`
`24`	Non-member small order pays	`shipping_fee(20.0, False) == 7.99`
`28`	Receipt formatting	`format_receipt(12.5) == "Total due: $12.50"`

```
pytest tests/test_checkout_v2.py --cov=checkout_app \
  --cov-branch --cov-report=term-missing
```

```
checkout.py    19    0    10    0    100%
9 passed
```

100%

statements and branches

What 100% does not mean

LIVE DEMO 7
no-assert + mutant

! No assertions

test_checkout_noassert.py also reaches 100% statement and branch coverage, with no assert at all.

```
pytest \
 tests/test_checkout_noassert.py \
 --cov=checkout_app --cov-branch
-> 100%, 1 passed
```

! Short-circuit

Line 22: every v2 test uses a member or a $20 order, so amount >= 50 is never true. Free shipping over $50 is untested.

```
if is_member or amount >= 50:
#  True ->|  never evaluated
#  False -> False
```

! Boundaries

Change line 22 to amount > 50. The v2 suite still passes at 100%. Only shipping_fee(50.0, False) catches it.

```
# edit line 22: >= 50  ->  > 50
-> 9 passed, 100%
# undo the edit afterwards
```

Try it: beyond 100%

TRY IT 4
students · 6 min

1. Write tests/test_beyond.py: make amount >= 50 true at least once, and test the $50 boundary.

```
from checkout_app.checkout import shipping_fee

def test_large_order_ships_free(): ...
def test_exactly_50_ships_free(): ...
```

2. Run v2 plus your file. Coverage stays at 100%. What did your tests add?

```
pytest tests/test_checkout_v2.py \
  tests/test_beyond.py \
  --cov=checkout_app --cov-branch
```

3. Change line 22 to amount > 50 and re-run. Which test fails? Undo the edit and delete your file.

```
pytest tests/test_checkout_v2.py tests/test_beyond.py
git checkout -- checkout_app/checkout.py
rm tests/test_beyond.py
```

Making the settings permanent

LIVE DEMO 8
config file

```
[tool.coverage.run]
branch = true
source = ["checkout_app"]

[tool.coverage.report]
show_missing = true
fail_under = 90
```

In a real project: pyproject.toml. The demo uses coverage_settings.toml, so earlier runs were not already in branch mode.

```
coverage run --rcfile=coverage_settings.toml \
  -m pytest tests/test_checkout_v1.py
coverage report --rcfile=coverage_settings.toml
```

```
checkout.py   19   5   10   3   66%   8, 16-17, 24, 28
Coverage failure: total of 66 is less than fail-under=90
```

exit 2

the report command fails below 90%

Quick reference

Command	Result on this demo
`coverage run -m pytest ...  +  coverage report`	81% (test files wrongly included)
`--source=checkout_app  +  report -m`	74% statement; missing lines listed
`--branch`	66%; adds Branch and BrPart columns
`coverage html`	Annotated report in htmlcov/
`pytest --cov=checkout_app --cov-branch --cov-report=term-missing`	Same 66%, in one command
`--cov-fail-under=80`	Tests pass, command fails (exit 1)

Practice question

TRY IT
think, then vote · 2 min

After running `test_checkout_coupons_only.py`, the report shows `16->18` in the Missing column.

Which single added test removes 16->18 from the report?

A
```
assert apply_coupon(3.0, "FLAT5") == 0.0
```

B
```
assert apply_coupon(100.0, "BOGUS") == 100.0
```

C
```
assert apply_coupon(50.0, "SAVE10") == 45.0
```

D
```
assert shipping_fee(20.0, False) == 7.99
```

Answer: B

LIVE DEMO 9
check the answer

B `assert apply_coupon(100.0, "BOGUS") == 100.0` → **`Missing: 5-10, 22-24, 28`**

+ **Why** Only an unrecognized code makes line 16 false and jump straight to line 18. Cover rises from 41% to 45%; BrPart drops to 0.

× **A, C** Repeat outcomes the suite already took (FLAT5 true, SAVE10 true).

× **D** Tests a different function: it removes lines 22-24, not the arrow.

Key takeaways

1. Measure branches, and only your application code (--branch, --source).
2. Read the Missing column, not the percentage: every entry is an untested behavior.
3. An arrow a->b asks: what input sends line a straight to line b?
4. Close each gap with a behavior-level test that asserts something.
5. 100% shows what ran, not what was checked: assertions, conditions, boundaries.